RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

MODULE 4

Deep Learning Foundations

Understanding the Architecture, Mechanics, and Implementation of "Thinking" Machines.

PART 1

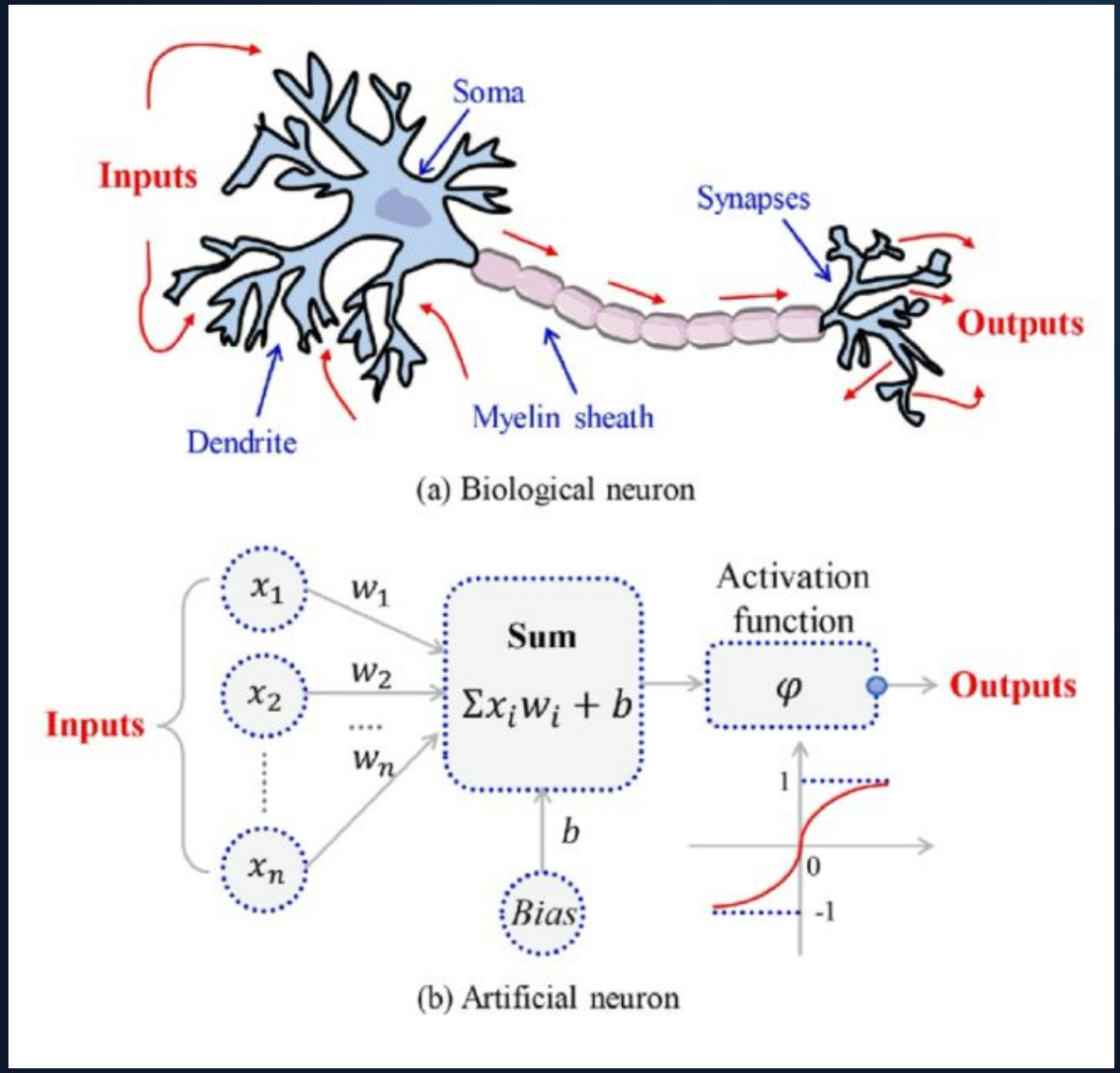
Architecture Basics

From Biological Inspiration to Artificial Neural Networks

Biological vs. Artificial

The human brain contains roughly 86 billion interconnected neurons. They communicate via electrical and chemical signals across synapses. When signals exceed a certain threshold, the neuron "fires."

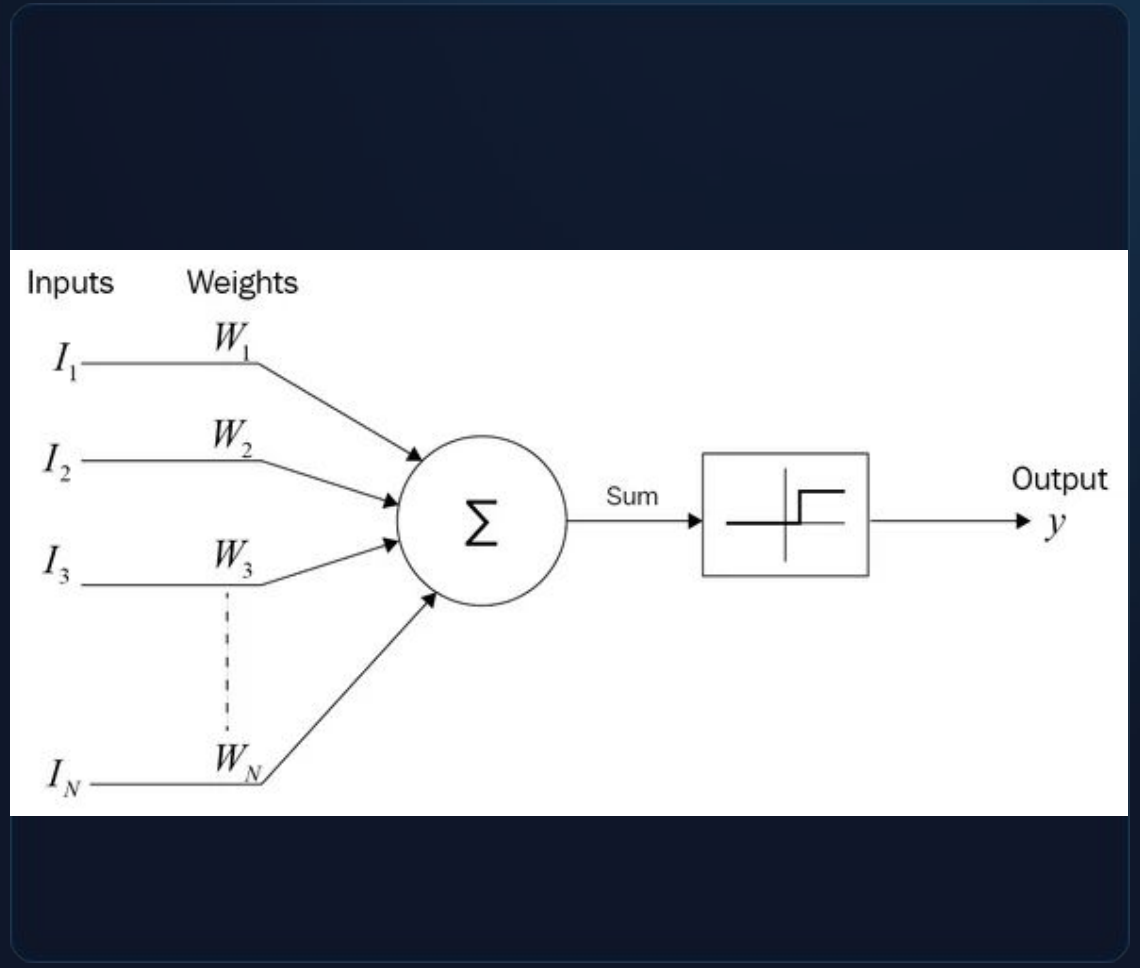
- **Artificial Neural Networks (ANNs)** mimic this structure mathematically.
- **Nodes** act as biological neurons.
- **Connection Weights** act as the synapses.
- **Activation Functions** simulate the "firing" threshold.



The First Step: McCulloch-Pitts (1943)

Created by Warren McCulloch and Walter Pitts, this was the absolute first mathematical model of a biological neuron.

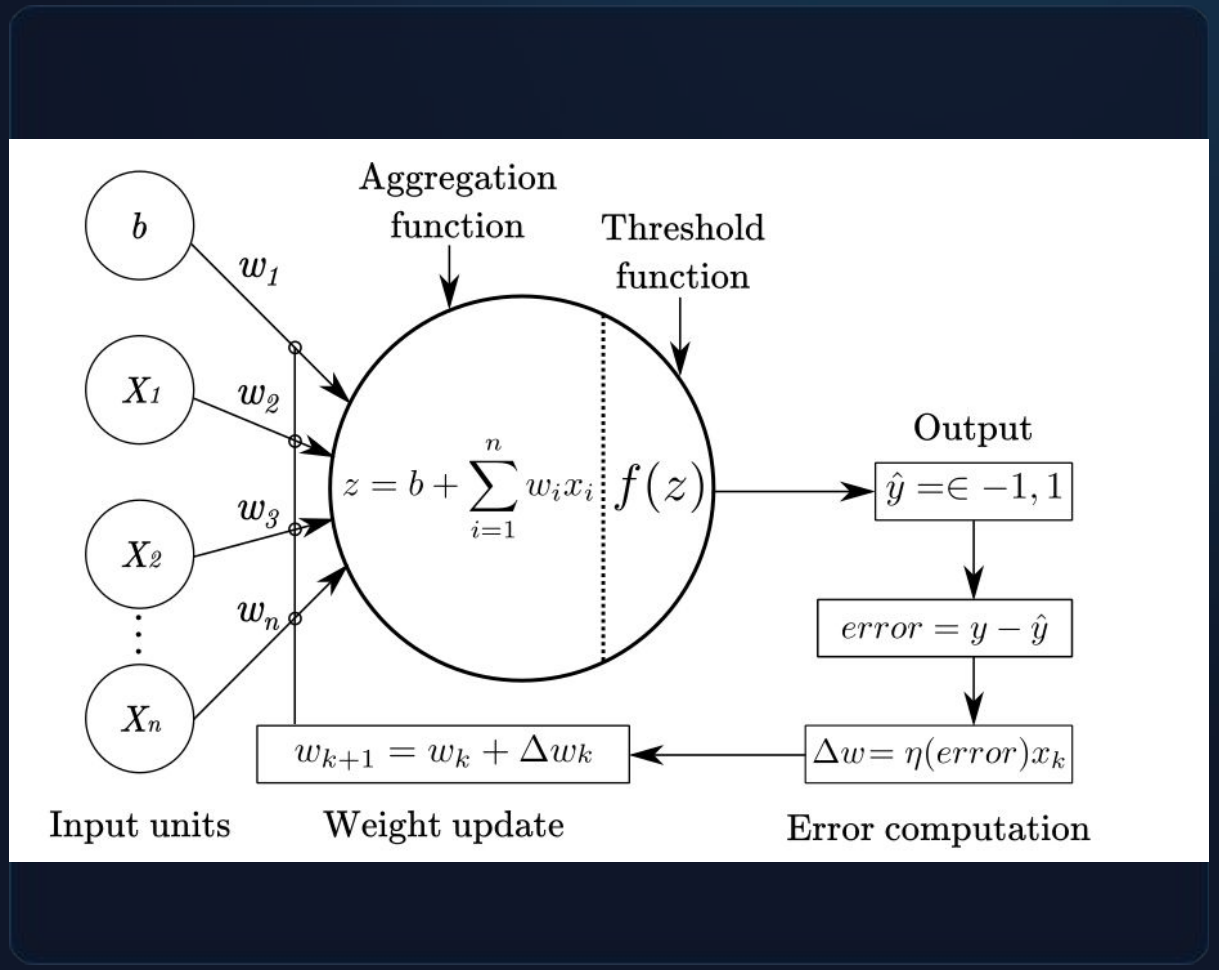
- **Binary Operation:** It only accepted binary inputs (0 or 1) and output a binary result.
- **Equal Weights:** All incoming signals were treated equally; there were no separate weights to prioritize certain inputs.
- **The Fatal Flaw: No Learning.** The MCP neuron could not learn from data. Engineers had to manually hand-calculate and hardcode the threshold values to make it solve logic gates.



The Perceptron (1957)

In 1957, Frank Rosenblatt introduced a massive leap forward: **Learnable Weights**. The Perceptron solved the MCP's fatal flaw by automatically learning its own parameters through trial and error.

- **Inputs (x):** The data features fed into the node.
- **Weights (w):** Determine the importance of each input signal. (These start random!)
- **Bias (b):** Shifts the activation threshold, allowing the model to fit data not passing through the origin.



Perceptron Math Model

The core operation is a linear combination followed by an activation step.

1. The Weighted Sum (z)

Calculates the dot product of inputs and weights, then adds the bias.

$$z = \sum_{i=1}^{n} (w_i \cdot x_i) + b$$

2. The Activation (y)

Passes the weighted sum through a function **f()** (like a Step or Sigmoid function) to get the final output.

$$y = f (z)$$

How Does It Learn? (The Process)

A Perceptron doesn't magically know the perfect weights. It finds them using the **Perceptron Learning Rule**.

1. **Initialization (Random Guesses)**
   The machine starts by assigning random numbers (often zeros or tiny decimals) to the weights and the bias.

2. **Forward Pass (Make a Prediction)**
   It takes the first row of training data, multiplies by the current random weights, and outputs a prediction.

3. **Calculate Error**
   It compares its prediction to the actual correct answer. If correct, Error = 0. If wrong, Error = +1 or -1.

4. **Update & Repeat (Epochs)**
   If wrong, it nudges the weights mathematically. It repeats this for every row, looping over the dataset (Epochs) until errors hit zero.

The Weight Update Rule

The Mathematical Nudge

If the Perceptron makes a mistake, it uses this exact formula to update each weight individually in Step 4:

$$w_{new} = w_{old} + (\eta \times Error \times Input)$$

Note: Error = (Actual Target - Predicted Output)

Deconstructing the Formula

- **Error = 0:** If the prediction is right, the addition term becomes zero. The weight stays exactly the same.
- **Error = ±1:** Determines the direction of the nudge (increase or decrease the weight).
- **Input:** The magnitude of the input feature scales the adjustment.
- **Learning Rate (η):** A tiny scalar (e.g., 0.1) that prevents wild overcorrections.

Solving Logic: AND Gate

After many Epochs of updating weights, the Perceptron finally converges on these **perfect, learned weights** for the **AND** logic gate.

`Final Learned Setup: w1 = 1 | w2 = 1 | b = -1.5 | f(z) = Step`

Input (x1)	Input (x2)	Weighted Sum: z = (x1·1) + (x2·1) - 1.5	Output: f(z)
0	0	0 + 0 - 1.5 = -1.5	0
0	1	0 + 1 - 1.5 = -0.5	0
1	0	1 + 0 - 1.5 = -0.5	0
1	1	1 + 1 - 1.5 = 0.5	1

Result: Output is 1 only when both inputs are 1. This data is **linearly separable** by a single straight line on a 2D graph.

Solving Logic: OR Gate

If we train the model on OR gate data instead, the Learning Rule automatically shifts the **bias parameter** to solve it.

`Final Learned Setup: w1 = 1 | w2 = 1 | b = -0.5 | f(z) = Step`

Input (x1)	Input (x2)	Weighted Sum: z = (x1·1) + (x2·1) - 0.5	Output: f(z)
0	0	0 + 0 - 0.5 = -0.5	0
0	1	0 + 1 - 0.5 = 0.5	1
1	0	1 + 0 - 0.5 = 0.5	1
1	1	1 + 1 - 0.5 = 1.5	1

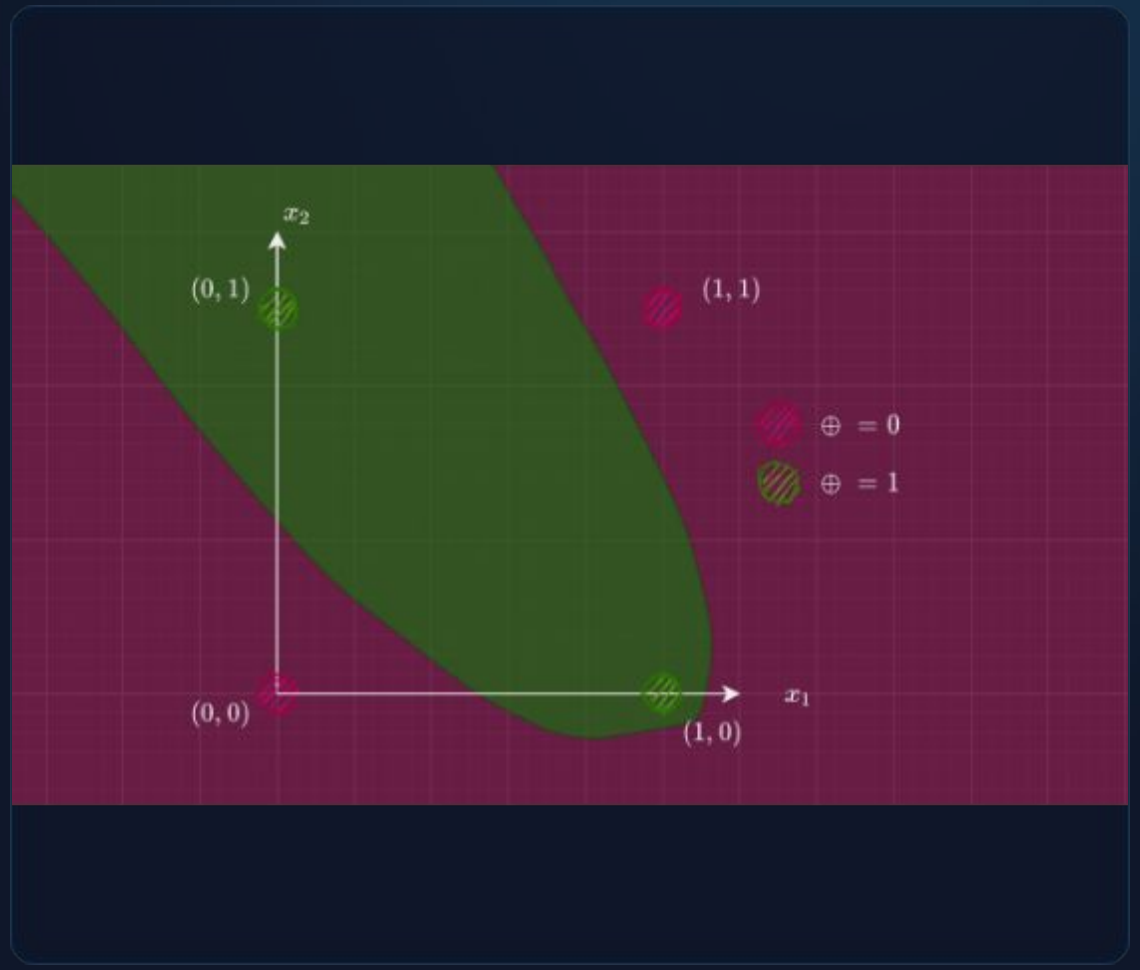
The Power of the Bias: By updating the bias from -1.5 to -0.5, the decision boundary line shifted, separating the 0s and 1s perfectly.

Limitation: The XOR Problem

A single perceptron creates a single linear decision boundary. It inherently cannot solve problems where the data is not linearly separable, no matter how many Epochs it trains for.

- ✓ **Linearly Separable:** AND/OR gates can be cleanly separated by drawing a single straight line on a graph.
- ✗ **Non-Linear (XOR):** The Exclusive-OR (XOR) gate outputs True only when inputs differ (0,1 or 1,0).

Graphing XOR results in points that **cannot** be divided by a single straight line. A single perceptron fails here, proving the need for deeper network architectures.



Multi-Layer Perceptron (MLP)

To solve complex, non-linear problems like XOR, we stack perceptrons into layers, creating a Feedforward Artificial Neural Network.

Input Layer

Receives the initial raw data. The number of nodes here exactly equals the number of features in your dataset.

Hidden Layers

Where the mathematical computation happens. "Deep" learning officially begins when a network contains more than one hidden layer.

Output Layer

Produces the final prediction. Its structure depends on the task (e.g., 1 node for regression, N nodes for multi-class classification probabilities).

PART 2

The Engine

Forward Propagation, Backward Propagation, and Optimizers

Forward Propagation

Forward propagation is the sequential process of pushing input data through the network layers to generate a final prediction.

1

Input Initialization

Data features are fed into the input layer nodes to begin the cycle.

2

Linear Transformation

Inputs are multiplied by connecting weights and summed together with a bias term at every single node in the subsequent hidden layer.

3

Non-linear Activation

The resulting weighted sum is passed through an activation function to determine the node's output, which then becomes the input for the next layer.

Forward Prop: Numerical

Calculating the output of a single hidden node during forward propagation.

The Setup

Assume we have the following inputs, weights, bias, and activation function:

```
Input Vector X = [2.0, 3.0]
Weight Vector W = [0.5, -0.2]
Bias b = 0.1
Activation = ReLU(z) = max(0, z)
```

The Calculation

Step 1: Weighted Sum (z)

```
z = (2.0 * 0.5) + (3.0 * -0.2) + 0.1
z = 1.0 - 0.6 + 0.1 = 0.5
```

Step 2: Activation Output (a)

```
a = ReLU(0.5) = max(0, 0.5) = 0.5
```

The node outputs 0.5 to the next layer.

Backward Propagation

If forward propagation is "making a guess," backward propagation is "learning from mistakes." It is the Deep Learning version of the Perceptron Learning Rule.

Blame Assignment

- **Error Calculation:** Compare the network's final prediction against the actual true value using a Loss Function.
- **Working Backwards:** Traverse backward from the output layer to the input layer.
- **Determine Contribution:** Calculate exactly how much each specific weight and bias in the network contributed to the final error.
- **Update:** Adjust the weights slightly in the opposite direction of the error gradient.

The Chain Rule (Math)

Backpropagation is purely an application of the **Chain Rule** from calculus to compute the partial derivative of the Loss (L) with respect to any weight (w).

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}$$

Deconstructing the Rule

- **∂L / ∂a:** How the loss changes as the activation changes.
- **∂a / ∂z:** The derivative of the activation function itself.
- **∂z / ∂w:** How the weighted sum changes with respect to the weight (which mathematically evaluates to just the input 'x').

Optimizing: Gradient Descent

Once Backprop finds the gradient (slope of error), an Optimizer decides **how** to update weights. Equation: w_new = w_old - (Learning_Rate × Gradient)

Stochastic GD (SGD)

Updates weights based on a single training example or small batch. While effective, it can be very noisy and bounce around the ravines of the loss landscape, slowing convergence.

SGD with Momentum

Adds a fraction of the previous weight update to the current one. Like a heavy ball rolling down a hill, it gains speed and plows right through small local minima.

Adam Optimizer

Adaptive Moment Estimation. Computes individual learning rates for different parameters. It is the **industry standard** today due to fast, robust convergence.

PART 3

Architecture Decisions

Activation Functions & Loss Functions

Why Activation Functions?

The Need for Non-Linearity

Without activation functions, a neural network is nothing more than a series of linear transformations (basic matrix multiplications).
If you stack 100 hidden layers with no non-linear activation function, the entire network mathematically collapses down into a **single-layer linear regression model**.

"Activation functions allow networks to learn complex, non-linear mappings—like identifying the curves of a face, translating languages, or steering a car."

Activation Functions List

Function	Range & Shape	Common Usage & Characteristics
Sigmoid	(0, 1) | S-curve	Binary classification output layer (Predicting Yes/No). Suffers from the vanishing gradient problem in deep networks.
Tanh	(-1, 1) | Zero-centered S-curve	Used in hidden layers of Recurrent Neural Networks (RNNs). Also prone to vanishing gradients.
ReLU	[0, ∞) | max(0, x)	**Default choice** for hidden layers. Solves vanishing gradients and is computationally very cheap.
Softmax	(0, 1) | Probabilities	Output layer for **Multi-class Classification**. Turns a vector of values into probabilities that sum to exactly 1.

Measuring Error: Loss Functions

Loss functions quantify exactly how far off the network's predictions are from the ground truth.

Mean Squared Error (MSE)

Calculates the average of the squared differences between predicted and actual values. It heavily penalizes large errors.

- **Primary Task:** Regression
- **Example:** Predicting continuous numerical outputs like house prices or temperature.

Cross-Entropy Loss

Measures performance of a classification model whose output is a probability value (0 to 1). Heavily penalizes confident but incorrect predictions.

- **Primary Task:** Classification
- **Example:** Categorizing images (e.g., Cat vs. Dog vs. Bird).

PART 4

Designing & Training

Bringing the Code to Life with Keras

Shallow vs. Deep Networks

Shallow Networks: Usually contain 1 hidden layer. While theoretically capable of approximating any function, they require an unmanageable, exponentially large number of nodes in that single layer to match deep network performance.

Deep Networks: Contain multiple hidden layers that learn hierarchical feature representations.

- Layer 1 extracts edges.
- Layer 2 extracts shapes.
- Layer 3 extracts complete faces.

Concept: Depth allows for feature composition.

Aspect	Shallow Learning	Deep Learning
Model Complexity	Limited depth, typically 1-2 layers.	High depth, consisting of multiple hidden layers.
Feature Engineering	Manual feature engineering required.	Automated feature extraction through hidden layers.
Data Requirement	Works well with small datasets.	Requires large datasets for effective learning.
Computational Cost	Low; can run on standard CPUs.	High; often requires GPUs or TPUs for training.
Performance on Complex Data	Limited for complex, non-linear relationships.	Excellent at capturing complex, non-linear patterns.
Interpretability	High; easy to understand model decisions.	Low; often considered a black-box model.
Training Time	Fast training due to fewer parameters.	Longer training time due to the complexity of the network.
Scalability	Suitable for smaller datasets and low-dimensional data.	Scalable with large datasets and high-dimensional data.

Keras: Network Architecture

Building a Deep Neural Network requires defining the sequence of layers, node counts, and activations.

```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    # Input Layer (Flattening 28×28 images)
    layers.Flatten(input_shape=(28, 28)),

    # Deep Hidden Layers with ReLU activation
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),

    # Output Layer with Softmax for 10-class
probability
    layers.Dense(10, activation='softmax')
])
```

Keras: The Training Loop

To train the model, we compile it with an Optimizer and Loss function, then execute the epochs.

```
# 1. Compile the Model
model.compile(
    optimizer='adam', # The engine driving the updates
    loss='sparse_categorical_crossentropy', # Error metric
    metrics=['accuracy'] # Human-readable evaluation
)

# 2. Train the Model (The Loop)
history = model.fit(
    X_train_data,
    y_train_labels,
    epochs=50,           # Full passes over dataset
    batch_size=32,       # Samples per gradient update
    validation_split=0.2 # Hold out data to check
overfitting
)
```

PART 5

Regularization

Preventing Neural Networks from Memorizing the Data

The Overfitting Problem

Deep networks have millions of parameters and can easily memorize training data rather than learning the underlying generalized patterns.

Failure to Generalize

- **Symptom:** Training loss steadily approaches zero, but Validation loss suddenly begins to climb upward.
- **Result:** Poor performance. The model fails completely when exposed to real-world, unseen data.
- **Solution:** Regularization. Artificially constraining the network to force generalized learning.

L1 and L2 Regularization

These techniques add a penalty term to the Loss Function based on the magnitude of the weights. We force the optimizer to minimize the error *AND* keep the weights small.

L1 Regularization (Lasso)

Adds the absolute value of weights to the loss. It drives less important feature weights exactly to zero, acting as automatic feature selection.

$$\text{Penalty} = \lambda \sum |w_i|$$

L2 Regularization (Ridge)

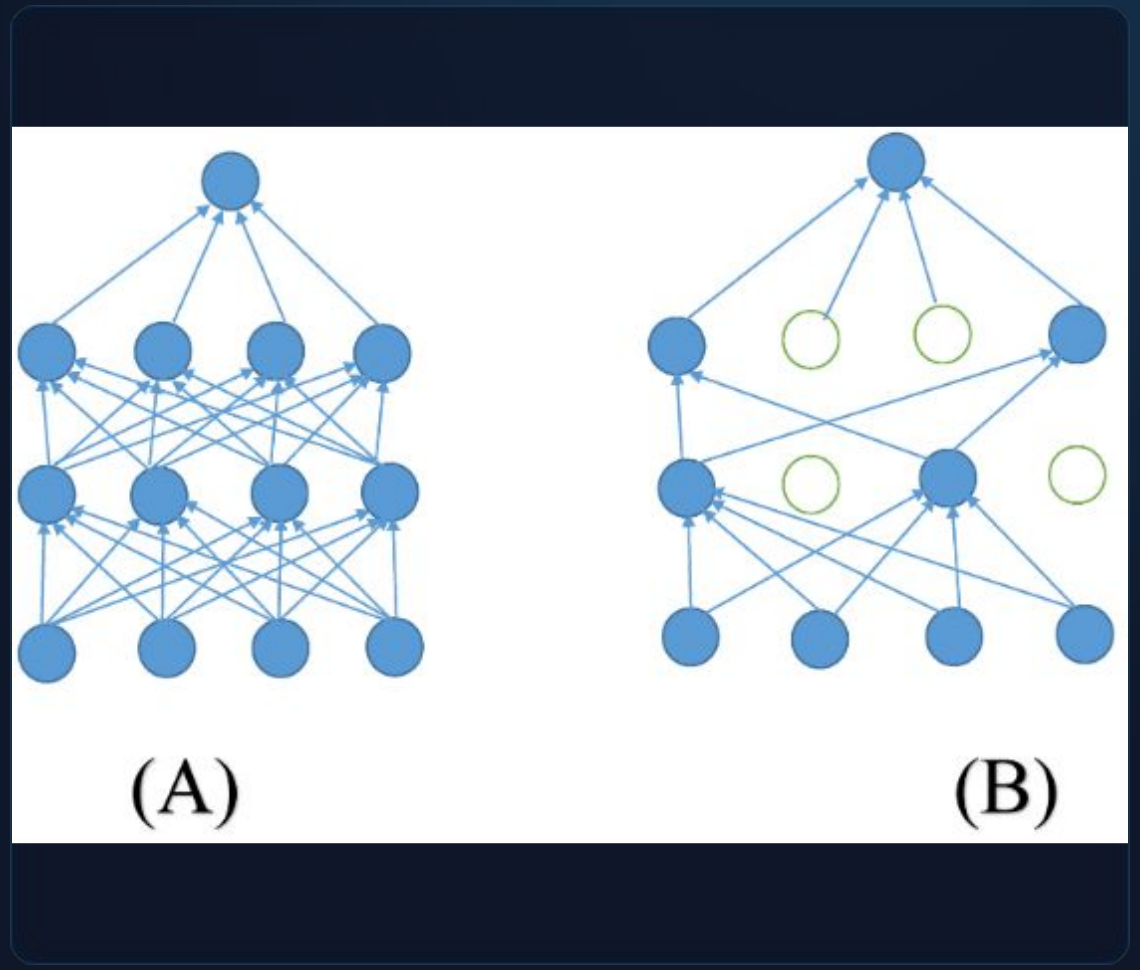
Adds the squared magnitude of weights to the loss. Shrinks weights evenly towards zero without eliminating them completely. Prevents any single feature from dominating.

$$\text{Penalty} = \lambda \sum w_i^2$$

Dropout Regularization

A powerfully simple regularization technique specifically designed for deep neural networks.

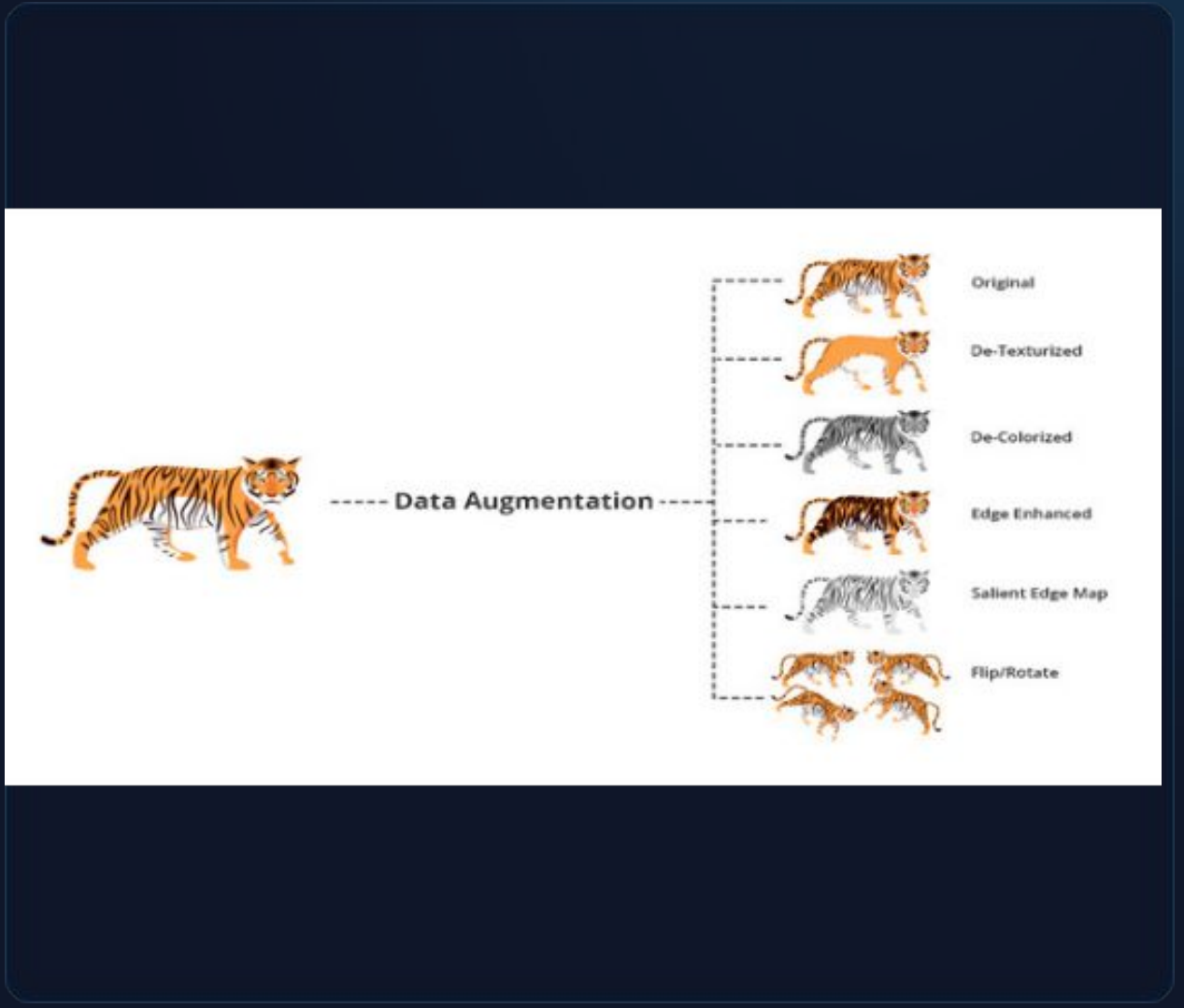
- **Mechanism:** During training, randomly "drop" (ignore) a percentage of neurons in a layer (e.g., 50%).
- **Effect:** Forces the network to distribute learning across multiple paths. No single neuron can rely on specific neighbors.
- **Result:** Creates an "ensemble" effect within a single network, massively reducing overfitting.



Data Augmentation

The absolute best way to prevent overfitting is having more training data. If you don't have it, **generate it**.

- **Concept:** Apply random transformations to existing data to create variations during training.
- **Techniques (Images):** Random rotations, zooming, flipping horizontally, adjusting brightness.
- **Goal:** Forces the model to recognize the core object regardless of its position, lighting, or orientation.



PART 6

Beyond MLPs

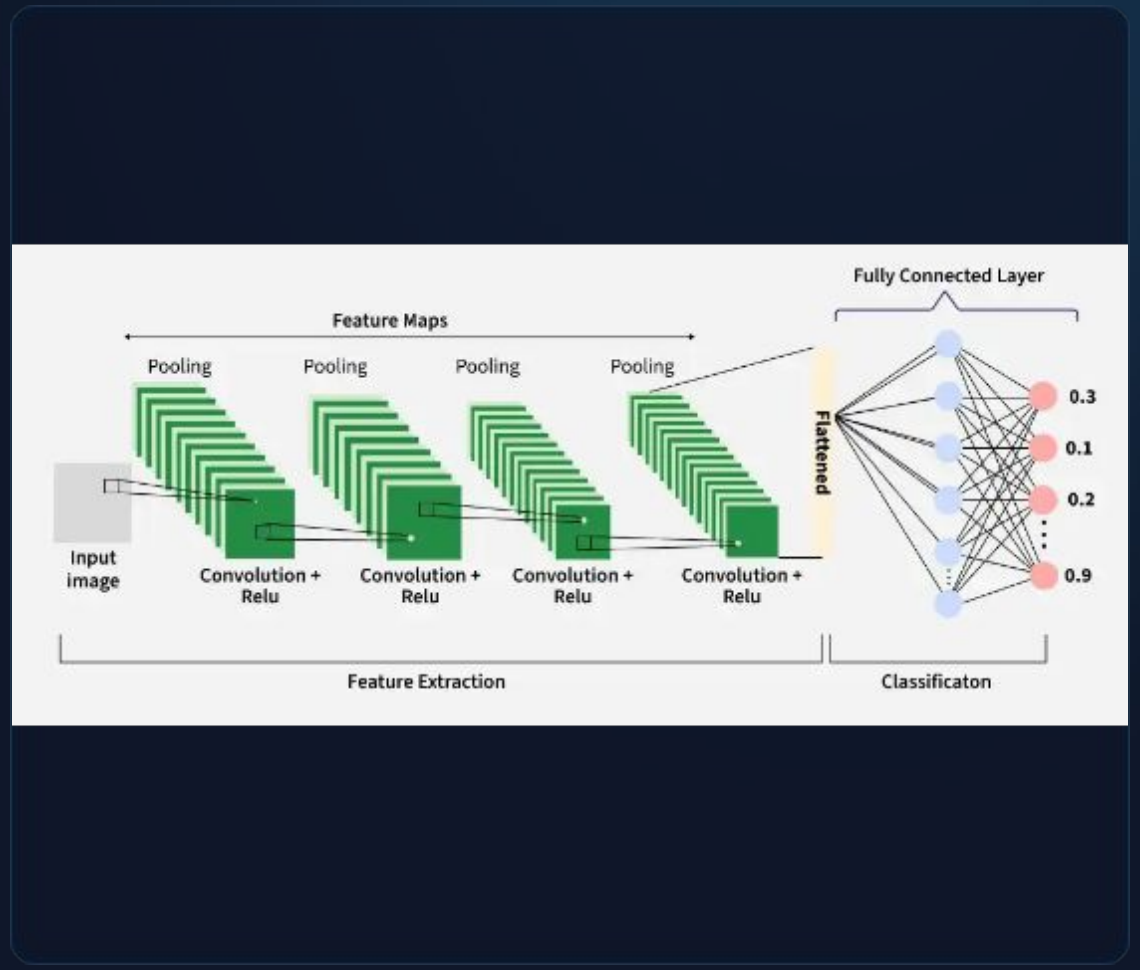
Advanced Architectures: CNNs and RNNs

Convolutional Neural Networks

The Visionaries of AI

CNNs are specialized for processing grid-like topology, primarily pixels in an image. They use sliding filters (convolutions) to automatically extract spatial features like edges and textures without flattening the image first.

- **Image Classification:** Categorizing photos (medical scan analysis, facial recognition).
- **Object Detection:** Identifying and locating multiple items within a frame (autonomous driving).

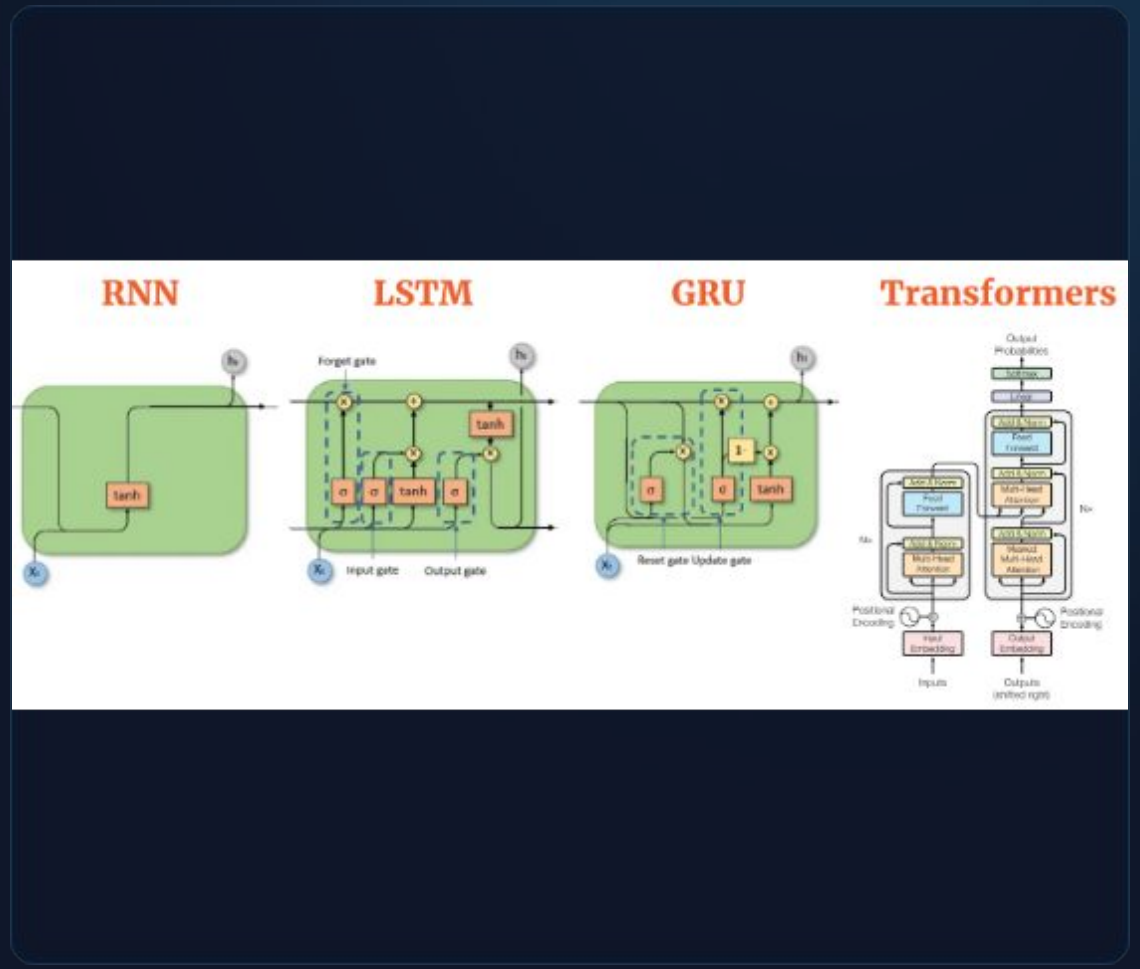


Recurrent Neural Networks

Masters of Sequence and Time

Unlike standard MLPs, RNNs contain internal loops, allowing information to persist. They have "memory" of previous inputs. **LSTMs** (Long Short-Term Memory) are advanced variants that remember long sequences.

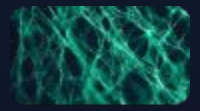
- **NLP:** Machine translation, sentiment analysis, text generation.
- **Speech Recognition:** Converting audio waves into text.
- **Time-Series:** Stock market, weather forecasting.



Questions?

End of Module 4: Deep Learning Foundations

Image Sources



https://plus.unsplash.com/premium_photo-1764695627178-a2888ec8dee9?fm=jpg&q=60&w=3000&ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1yZWxhdGVkfDI5fHx8ZW58MHx8fHx8

Source: unsplash.com

https://miro.medium.com/1*1Oh53dNdPITVnoOVGCCUFA.png

Source: medium.com

https://gamedevacademy.org/wp-content/uploads/2017/09/Single-Perceptron.png.webp

Source: gamedevacademy.org

https://towardsdatascience.com/wp-content/uploads/2020/11/1fgcoOFn-gfKnA_U5BOw-XA.png

Source: towardsdatascience.com

https://miro.medium.com/v2/resize:fit:1400/1*6osX0-t4tFz0BWPIS0GB9Q.png

Source: medium.com

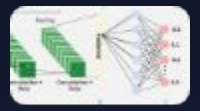
https://www.mdpi.com/electronics/electronics-12-03106/article_deploy/html/images/electronics-12-03106-g001.png

Source: www.mdpi.com

Image Sources

https://insights.daffodilsw.com/hs-fs/hubfs/Archna/Color_Space_Transformation.jpg?width=600&name=Color_Space_Transformation.jpg

Source: insights.daffodilsw.com



https://media.geeksforgeeks.org/wp-content/uploads/20260217113409515144/working_of_cnn__.webp

Source: www.geeksforgeeks.org

https://aiml.com/wp-content/uploads/2023/10/rnn-lstm-gru-transformers.png

Source: aiml.com